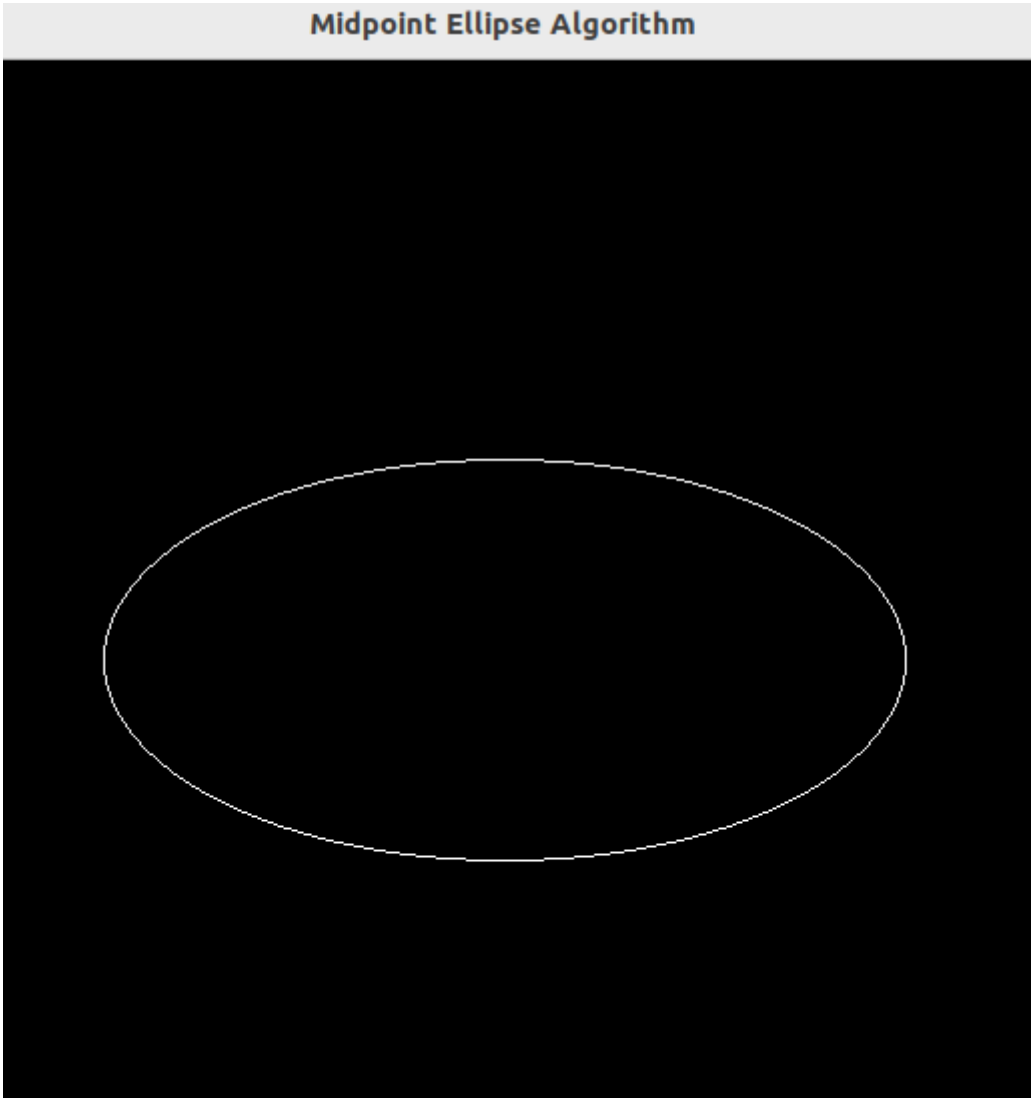
Lab 5

Q1. Implement Ellipse drawing algorithm.

```
#include <GL/glut.h>
#include <cmath>
#include <iostream>

int xc = 0, yc = 0;
int rx = 200, ry = 100;

void plotPoints(int x, int y)
{
   glVertex2i(xc + x, yc + y);
   glVertex2i(xc - x, yc + y);
   glVertex2i(xc + x, yc - y);
   glVertex2i(xc - x, yc - y);
}

void midpointEllipse()
{
   int x = 0;
   int y = ry;

   float dx, dy;
   float d1, d2;

   dx = 2 * ry * ry * x;
   dy = 2 * rx * rx * y;


   d1 = (ry * ry) - (rx * rx * ry) + (0.25 * rx * rx);

   glBegin(GL_POINTS);

   while (dx < dy)
   {
      plotPoints(x, y);

      if (d1 < 0)
      {
         x++;
         dx = dx + (2 * ry * ry);
         d1 = d1 + dx + (ry * ry);
      }
      else
      {
         x++;
         y--;
         dx = dx + (2 * ry * ry);
         dy = dy - (2 * rx * rx);
         d1 = d1 + dx - dy + (ry * ry);
```

```
        }
    }


    d2 = ((ry * ry) * ((x + 0.5) * (x + 0.5))) +
         ((rx * rx) * ((y - 1) * (y - 1))) -
         (rx * rx * ry * ry);

    while (y >= 0)
    {
        plotPoints(x, y);

        if (d2 > 0)
        {
            y--;
            dy = dy - (2 * rx * rx);
            d2 = d2 + (rx * rx) - dy;
        }
        else
        {
            y--;
            x++;
            dx = dx + (2 * ry * ry);
            dy = dy - (2 * rx * rx);
            d2 = d2 + dx - dy + (rx * rx);
        }
    }

    glEnd();
}

void display()
{
    glClear(GL_COLOR_BUFFER_BIT);

    glColor3f(1.0, 1.0, 1.0);
    midpointEllipse();

    glFlush();
}

void init()
{
    glClearColor(0.0, 0.0, 0.0, 1.0);
    glColor3f(1.0, 1.0, 1.0);

    glMatrixMode(GL_PROJECTION);
    gluOrtho2D(-400, 400, -300, 300);
}

int main(int argc, char** argv)
{
```

```
    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_SINGLE | GLUT_RGB);
    glutInitWindowSize(800, 600);
    glutCreateWindow("Midpoint Ellipse Algorithm");

    init();
    glutDisplayFunc(display);
    glutMainLoop();
    return 0;
}
```



Q2. Create 3 houses , Sun and Fence.

```
#include <GL/glut.h>
#include <cmath>

void drawSun(float cx, float cy, float radius)
{
    int segments = 100;
```

```
    glBegin(GL_LINE_LOOP);
        for(int i = 0; i <= segments; i++)
        {
            float theta = 2.0f * 3.14159f * i / segments;
            float x = cx + radius * cos(theta);
            float y = cy + radius * sin(theta);
            glVertex2f(x, y);
        }
    glEnd();

    glBegin(GL_LINES);
        for(int i = 0; i < 12; i++)
        {
            float angle = 2.0f * 3.14159f * i / 12;
            float x1 = cx + radius * cos(angle);
            float y1 = cy + radius * sin(angle);
            float x2 = cx + (radius + 20) * cos(angle);
            float y2 = cy + (radius + 20) * sin(angle);
            glVertex2f(x1, y1);
            glVertex2f(x2, y2);
        }
    glEnd();
}

void drawHouse()
{

    glBegin(GL_LINE_LOOP);
        glVertex2f(-40, -50);
        glVertex2f(40, -50);
        glVertex2f(40, 20);
        glVertex2f(-40, 20);
    glEnd();


    glBegin(GL_LINE_LOOP);
        glVertex2f(-50, 20);
        glVertex2f(0, 70);
        glVertex2f(50, 20);
    glEnd();


    glBegin(GL_LINE_LOOP);
        glVertex2f(-15, -50);
        glVertex2f(15, -50);
        glVertex2f(15, 0);
        glVertex2f(-15, 0);
    glEnd();
}

void drawFence()
```

```
{

   glBegin(GL_LINES);
      glVertex2f(-300, -80);
      glVertex2f(300, -80);

      glVertex2f(-300, -60);
      glVertex2f(300, -60);
   glEnd();


   glBegin(GL_LINES);
      for(int i = -300; i <= 300; i += 20)
      {
         glVertex2f(i, -100);
         glVertex2f(i, -50);
      }
   glEnd();
}

void display()
{
   glClear(GL_COLOR_BUFFER_BIT);


   glColor3f(0.0f, 0.0f, 0.0f);

   drawSun(200, 180, 30);

   glPushMatrix();
      glTranslatef(-150, 0, 0);
      drawHouse();
   glPopMatrix();

   glPushMatrix();
      drawHouse();
   glPopMatrix();

   glPushMatrix();
      glTranslatef(150, 0, 0);
      drawHouse();
   glPopMatrix();

   drawFence();

   glFlush();
}

void init()
{

   glClearColor(1.0f, 1.0f, 1.0f, 1.0f);
```

```
    glPolygonMode(GL_FRONT_AND_BACK, GL_LINE);

    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();
    gluOrtho2D(-300, 300, -150, 250);
}

int main(int argc, char** argv)
{
    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_SINGLE | GLUT_RGB);
    glutInitWindowSize(800, 600);
    glutCreateWindow("Three Houses - Outline Mode");

    init();
    glutDisplayFunc(display);
    glutMainLoop();
    return 0;
}
```